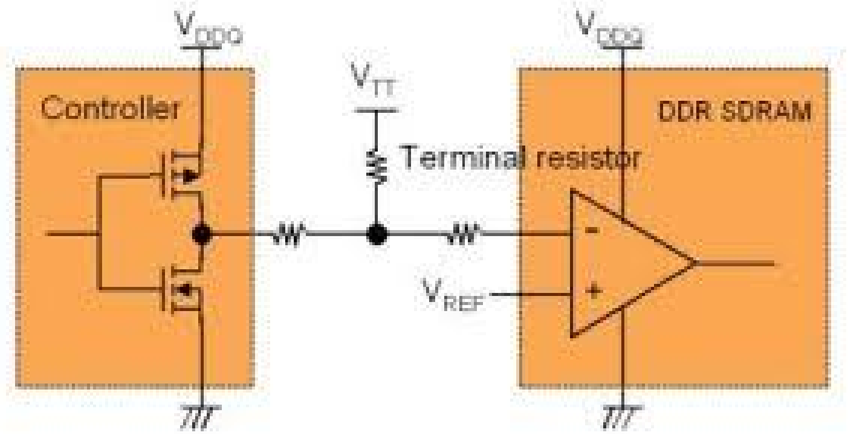
Output Capacitor and Decoupling Design for DDR Power Rails

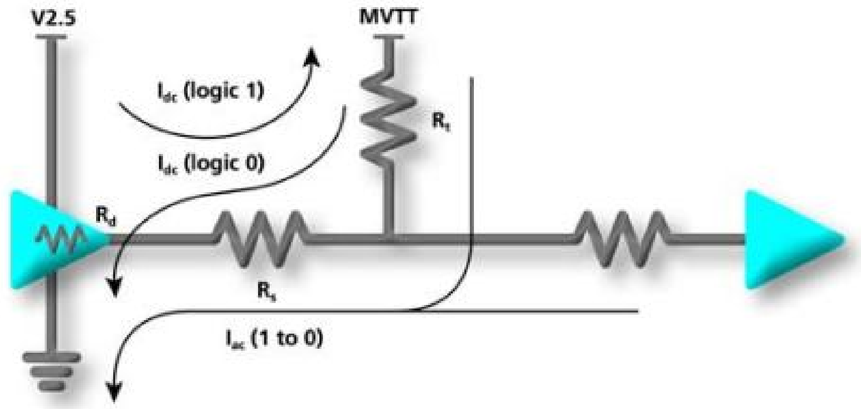
Unnath Chittimalla, IMT2023620

What is DDR?

Why optimize converters for DDR Rails?



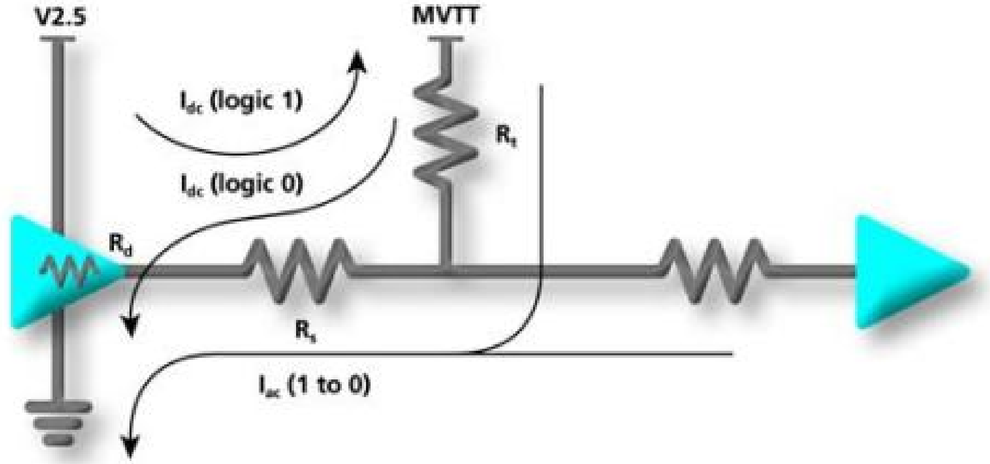
- Double Data Rate SDRAMs are the backbones of today’s computing.
- They allow memory transfer on both rising and falling edges of the clock. This behaviour “doubles” the data rate.
- This speed comes at the cost of increased switching activity leading to more short bursts of current demand especially affecting VDDQ and VTT rails.



How exactly do rail transients occur?

0 → 1 Transition.
Driver switches PMOS on, current is pulled from VDDQ (and its decoupling capacitors) immediately. This flows through PMOS driver and some of it is used to charge output capacitance (AC), and rest flows into the VTT path (DC).

1 → 0 Transition.
Driver switches NMOS on, output capacitor is discharged via driver into ground plane (AC) and the current also rushes from MVTT through Rt, Rd and to the ground plane (DC).



$$I_{dc} = \left(\frac{V_{DD} - MVTT}{R_s + R_d + R_t}\right) \quad \text{or} \quad I_{dc} = \left(\frac{2.5 - 1.25}{0 + 13 + 39}\right)$$

where:
MVTT = 1.25V
V_{DD} = 2.5V

$$I_{ac} = C\left(\frac{dv}{dt}\right) \quad \text{or} \quad I_{ac} = 30\left(\frac{1.875}{1.5}\right)$$

where:
C = load capacitance
dv = voltage range
dt = 10%–90% rise/fall time

Note: • DDR SDRAM worst-case input capacitance for DQ/DQS = 5pF
• Two double-sided DIMMs = 4 loads per net, 20pF. 2.5pF per inch for a 60 ohm PCB trace. 4 inches of trace would be 10pF. Trace capacitance plus DRAM capacitance is 30pF.
• Voltage switching range at load = 1.875V. 10% to 90% switching time, worst case = 1.5ns.

For this example:
I_{dc} = 24.0mA
I_{ac} = 37.5mA

Why is this dangerous?

Worst Case Burst
As many as 81 signals switch simultaneously on a single memory module. That is 81 times what we calculated earlier!

The Challenge
The challenge is not a lack of charge, but the parasitic inductance in the delivery path that creates a voltage drop (V = L.di/dt) during the transitions. We have to meet certain rail tolerance levels to satisfy setup and hold time constraints in the circuit.

Assume tolerance for MVTT rail = +-100mV and V2.5 rail = +-200mV

The Challenge
MVTT Rail is kept around VDDQ/2 = 1.25V.
If you consider a 0603 capacitor package and add in the inductance from vias, the parasitic inductance = 1.82nH, or any other capacitor with huge parasitic inductance.

- Number of nets switching = 109
- Current per net (di) for MVTT = 2*I_dc = 0.048A
- Switching time (dt) = 1.5ns (10% to 90%)

$$V_{noise} = L \cdot \frac{N \cdot di}{dt}$$

$$V_{noise} = (1.82 \times 10^{-9}\ \text{H}) \cdot \frac{109 \cdot 0.048\ \text{A}}{1.5 \times 10^{-9}\ \text{s}}$$

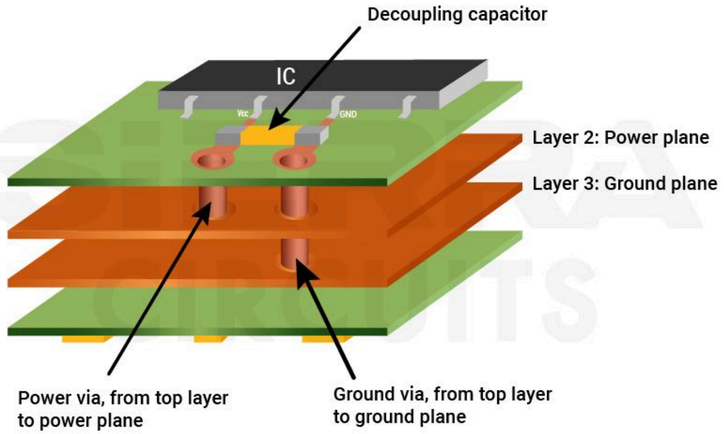
$$V_{noise} = 1.82 \cdot \frac{5.232\ \text{A}}{1.5\ \text{s}}$$

$$\boxed{V_{noise} \approx 6.35\ \text{V}}$$

How to design around this?

Calculate maximum impedance for each rail based on tolerance levels.

This gives us an idea of how the decoupling capacitors must be placed or how many to use in parallel so that the inductance criteria and rail tolerance is not violated. Placement of rails in different layers affects via length which changes the effective inductance as well.



Estimating parasitic inductances.

This gives us an idea of how the decoupling capacitors must be placed or how many to use in parallel so that the inductance criteria and rail tolerance is not violated.

For this example:
0603 cap was used with a package inductance of 0.87nH

$0603\ L_{eq} = L_{package} + L_{via}$

$0603\ L_{eq} = 0.87nH + 0.948nH = 1.82nH$

$N_{cap} = L_{eq}/L_{max}$

For this example:
MVTT $N_{cap} = 1.82nH/0.029nH = 63$
V2.5 $N_{cap} = 1.82nH/0.073nH = 25$

$$L_{via} = 5.08h\left[\ln\left(\frac{4h}{d}\right)+1\right] \text{ or } L_{via} = 5.08h\left[\ln\left(\frac{4(0.050)}{0.013}\right)+1\right]$$

Where:
h = via length
d = via diameter

For this example:
- 0.050" board
- 0.013" via
- $L_{via} = 0.948nH$

Note: Via sizes are different for the motherboard and DIMM module. For simplification we are using the same values for both.

$$L_{max} = \frac{V(dt)}{N(di)} \text{ or } L_{max} = \frac{0.1(1.5\times10^{-9})}{109(2\times0.024)}$$

Where:
V = MAX allowable voltage drop
dt = 10%–90% switching time
di = current per net
N = number of nets switching simultaneously = 109

Note:
- For di use 2 x Idc for MVTT and Iac for V2.5
- Tolerance specification for MVTT is ±100mV
- Tolerance specification for V2.5 is ±200mV

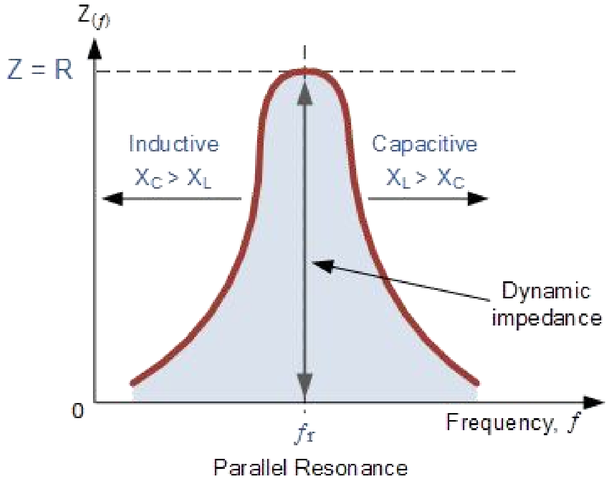
For this example:
MVTT $L_{max} = 0.029nH$
V2.5 $L_{max} = 0.073nH$

More output capacitors in parallel always better?

Anti Resonance Tanks.

Blindly adding different capacitor values like 100uF + 0.1uF does not always improve the power delivery network.

At specific frequencies, a parasitic inductance of one capacitor resonates with capacitance of another, creating impedance spikes which introduces more noise into the rails. Low ESR values amplify this further, while high ESR values minimize it but can contribute to DC losses and constant voltage droop.



Parallel Resonance

How to analyse this?

Sweeping the decoupling network over different frequencies and keeping track of the Self Resonant Frequencies of caps used from their datasheets will help in identifying problematic tanks. ESR values have to be looked after as well.

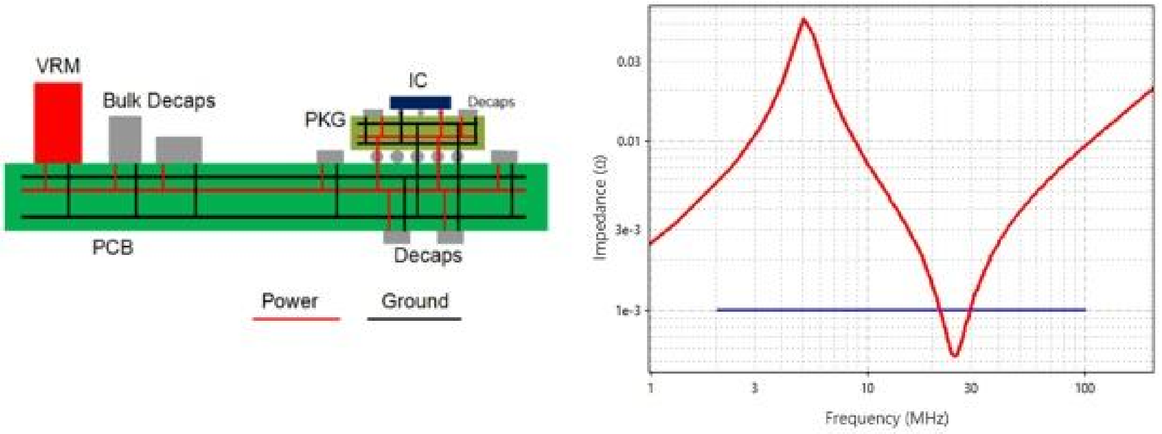
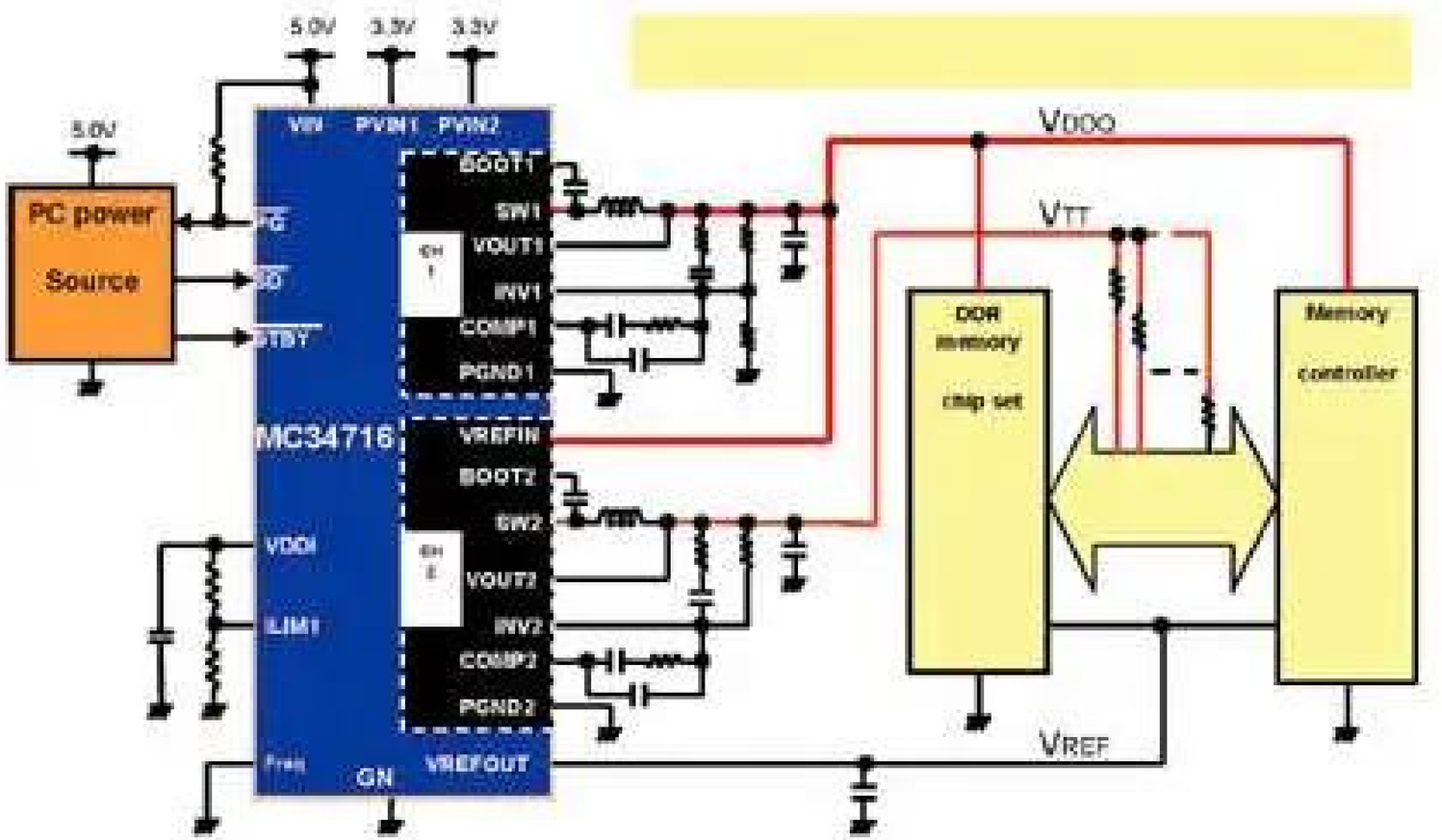


Figure 2 Left panel: Cross-section of a typical power rail layout for an IC fed by a VRM (voltage regulator module). The power rail feeds the IC through its package (PKG). Right panel: Power rail impedance as seen from the memory controller IC in a post-layout DDR-4 design (red) compared to the target impedance (blue). Source: [4]

Role of Buck Converters - Single Chip DDR (MC34716 IC)



Rails from the converter

- SW1 provides VDDQ (V2.5)
- SW2 provides VTT (1.25V)

- VREF is assumed to be a reference voltage also at 1.25V.

- The reason VREF is seperate from VTT is because it is a low current source / sink. It is used as input to error amplifiers.

- VTT needs to both source and sink currents while VDDQ only sources current.

LTSpice Modelling and Math Strategy

Behvaioral Load Model
Use PWL current sources to simulate the memory bus. Commercial buck converters will be used to ensure reproducibility of results.

If memory is 333MHz, we have a clock pulse every 3ns so we need to create current pulses every 1.5ns.

Violation Checks
We monitor the rails VTT, VDDQ (and VREF) to see if they fail the tolerance test given an output capacitor decoupling network.

Ablation Studies

Genetic Algorithm Approach:
Instead of guessing capacitor values, we can use a genetic algorithm to try out different combinations of capacitor values along with their ESR / ESL values to find combinations that minimize anti-resonance peaks (especially at 333MHz/666MHz).

Layout Strategies:
Minimize the number of capacitors required in parallel by eliminating the inductance caused by vias.

Results - Setup

ddr4_params.yaml

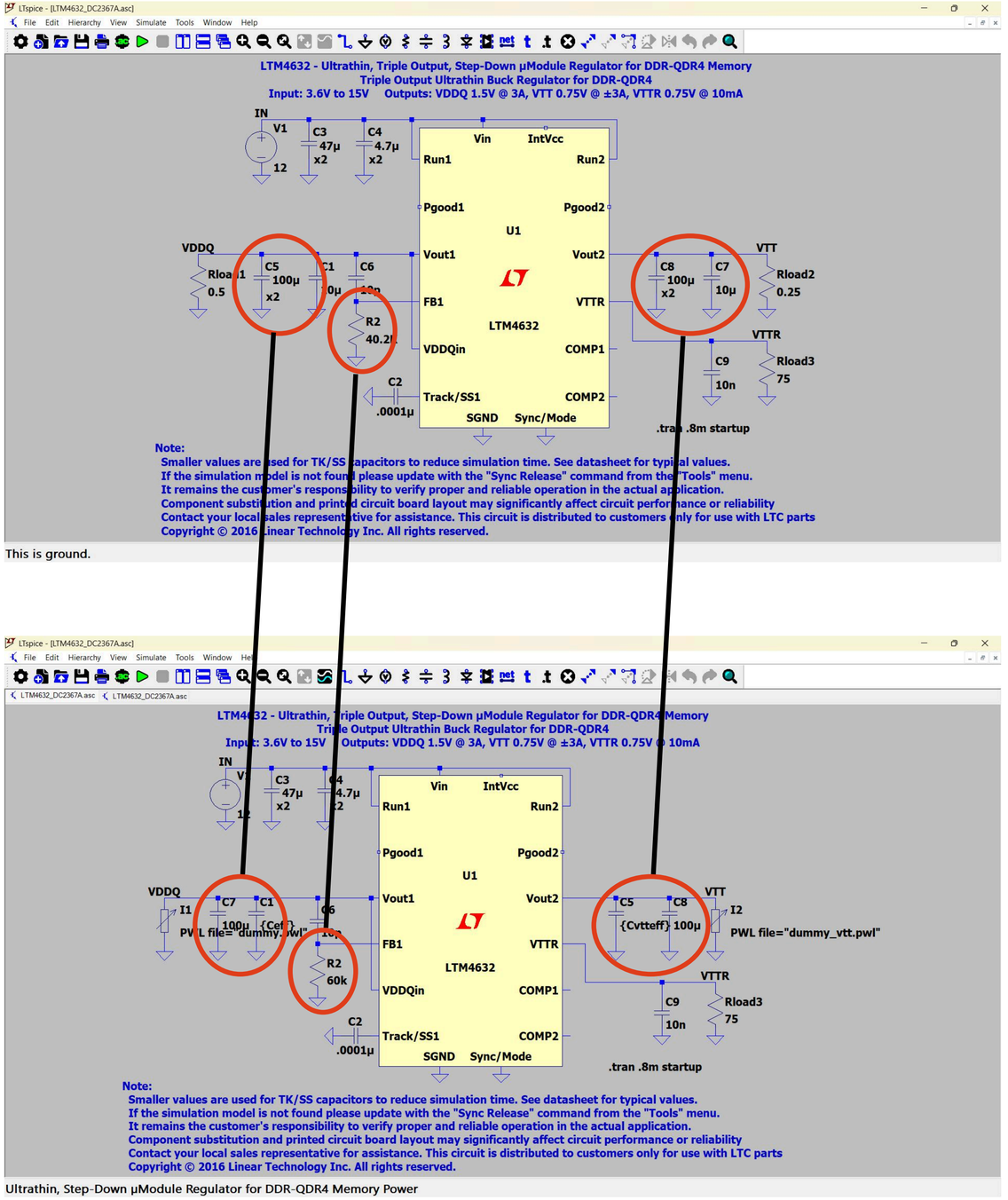
ddr4-pdn-optimizer > config > ddr4_params.yaml

```yaml
ddr4:
  vddq_v: 1.2
  vddq_tolerance_v: 0.06
  vpp_v: 2.5
  odt_ohm: 48
  chips_per_dimm: 8
  max_simultaneous_nets: 81
  speed_grade: DDR4-3200
  clock_mhz: 1600
  switching_time_ns: 0.2
  vtt_v: 0.6
  vtt_tolerance_v: 0.04
```

pdn_constraints.yaml

ddr4-pdn-optimizer > config > pdn_constraint

```yaml
rails:
  VDDQ:
    nominal_v: 1.2
    tolerance_v: 0.06
    lmax_pH: 5.93
    nmax_caps: 63
  VTT:
    nominal_v: 0.6
    tolerance_v: 0.04
    lmax_pH: 16.25
    nmax_caps: 20
via:
  board_thickness_in: 0.05
  via_diameter_in: 0.013
  l_via_nH: 0.948
```



Results - Setup

```
; rowhammer.asm — Adversarial double-sided row hammering workload
; Hammers two rows in the same bank with clflush + mfence between accesses.
; Maximises simultaneous switching events on VDDQ.
; Industry stress test used in reliability qualification.

section .data
    BUFFER_SIZE   equ 256 * 1024 * 1024  ; 256 MB
    ROW_OFFSET    equ 8192               ; Row size (8KB typical for DDR4)
    HAMMER_COUNT  equ 2000000            ; Number of hammer iterations

section .text
global _start
extern setup_mmap, exit_program

_start:
    ; Allocate 256MB buffer
    call   setup_mmap
    mov    r12, rax                  ; r12 = buffer base

    ; Select two rows in the same bank, separated by one row
    ; Row A: base + 0
    ; Row B: base + 2 * ROW_OFFSET (skip one row in between = double-sided)
    lea    r13, [r12]                ; row_a address
    lea    r14, [r12 + 2 * ROW_OFFSET] ; row_b address

    mov    rcx, HAMMER_COUNT         ; iteration count

    ; Align to prevent straddling cache lines
    align  16

.hammer_loop:
    ; Access row A
    mov    rax, [r13]        ; read row A (triggers ACT if not in row buffer)
    clflush [r13]            ; evict from all cache levels
    mfence                   ; serialise — ensure flush completes before next access

    ; Access row B (same bank, different row)
    mov    rbx, [r14]        ; read row B (forces PRE + ACT)
    clflush [r14]            ; evict from cache
    mfence                   ; serialise

    dec    rcx
    jnz    .hammer_loop

    call   exit_program
```

```
; random_access.asm — Pointer-chasing random access workload
; Models poor cache locality (database hash lookups, sparse NN inference).
; Builds a shuffled address array then pointer-chases through it.
; Every access is a guaranteed row miss in a random bank = max ACT/PRE.

section .data
    BUFFER_SIZE  equ 256 * 1024 * 1024  ; 256 MB
    STRIDE       equ 4096               ; Page-sized stride for maximum row misses
    NUM_ENTRIES  equ BUFFER_SIZE / STRIDE

section .text
global _start
extern setup_mmap, exit_program

_start:
    ; Allocate 256MB buffer
    call   setup_mmap
    mov    r12, rax           ; r12 = buffer base
    mov    r13, 0x12345678    ; Initial seed for LCG

    ; --- Phase 1: Build sequential index array ---
    ; Store pointers at each page boundary, initially sequential
    xor    rcx, rcx           ; i = 0
.build_loop:
    mov    rax, rcx
    inc    rax                ; next index
    cmp    rax, NUM_ENTRIES
    cmovge  rax, rcx          ; wrap last entry to 0 (but we fix below)
    lea    rdx, [r12 + rax * 8]   ; store pointer to next entry
    mov    [r12 + rcx * 8], rdx
    inc    rcx
    cmp    rcx, NUM_ENTRIES
    jb     .build_loop

    ; Make last entry point back to first
    lea    rax, [r12]
    mov    [r12 + (NUM_ENTRIES - 1) * 8], rax

    ; --- Phase 2: Fisher-Yates shuffle using rdrand ---
    mov    rcx, NUM_ENTRIES
    dec    rcx                ; i = N-1
.shuffle_loop:
    cmp    rcx, 0
    jle    .shuffle_done

    ; Get random-ish number via simple LCG instead of rdrand (for compatibility)
    ; rax = (rax * 6364136223846793005 + 1)
    mov    rax, r13           ; r13 holds the seed/state
    mov    rbx, 6364136223846793005
    mul    rbx
    inc    rax
    mov    r13, rax           ; save state

    ; j = rax % (i+1)
    xor    rdx, rdx
    mov    rbx, rcx
    inc    rbx
    div    rbx                ; rdx = rax % (i+1)

    ; Swap array[i] and array[j]
    mov    rsi, [r12 + rcx * 8]
    mov    rdi, [r12 + rdx * 8]
    mov    [r12 + rcx * 8], rdi
    mov    [r12 + rdx * 8], rsi

    dec    rcx
    jmp    .shuffle_loop

.shuffle_done:

    ; --- Phase 3: Pointer chase ---
    mov    rdi, [r12]         ; Start at first pointer
    mov    rcx, NUM_ENTRIES
    shl    rcx, 2             ; 4x iterations for sufficient trace length

.chase_loop:
    mov    rdi, [rdi]         ; Follow pointer chain
    dec    rcx
    jnz    .chase_loop

    call   exit_program
```

```
; streaming.asm — Sequential movnti (non-temporal store) workload
; Models a memory controller performing sequential prefetch fills.
; Uses cache-bypassing writes with 64-byte stride.
; Every access is a sequential row hit = best-case (minimum ACT/PRE rate).

section .data
    BUFFER_SIZE equ 256 * 1024 * 1024  ; 256 MB
    ITERATIONS  equ 4                  ; Number of full passes

section .text
global _start
extern setup_mmap, exit_program

_start:
    ; Allocate 256MB buffer
    call   setup_mmap
    mov    r12, rax                ; r12 = buffer base address

    ; Outer loop: multiple passes over the buffer
    mov    r13, ITERATIONS
.outer_loop:
    mov    rdi, r12                ; rdi = current write pointer
    lea    rsi, [r12 + BUFFER_SIZE]   ; rsi = end of buffer

    ; Inner loop: sequential movnti with 64-byte stride (cache-line sized)
.inner_loop:
    movnti  [rdi],     rax         ; Non-temporal store (bypasses cache)
    movnti  [rdi + 8],  rax
    movnti  [rdi + 16], rax
    movnti  [rdi + 24], rax
    movnti  [rdi + 32], rax
    movnti  [rdi + 40], rax
    movnti  [rdi + 48], rax
    movnti  [rdi + 56], rax

    add    rdi, 64                 ; Advance by one cache line
    cmp    rdi, rsi
    jb     .inner_loop

    sfence                         ; Ensure all NT stores are visible

    dec    r13
    jnz    .outer_loop

    call   exit_program
```

Results - Setup

Datasheet Constraints

```
[timing]
; DDR4-3200AA (CL22-22-22) — JEDEC standard timing
tCK=0.625
AL=0
CL=22
CWL=16
tRCD=22
tRP=22
tRAS=52
tRFC=560
tRFC2=416
tRFC4=256
tREFI=12480
tRPRE=1
tWPRE=1
BL=8
tCCD_L=8
tCCD_S=4
tRTP=12
tWTR_L=12
tWTR_S=4
tWR=24
tFAW=40
tRRD_L=8
tRRD_S=4
tXP=8
tXS=576
tCKE=8
tCKESR=9
```

```
[system]
channel_size=8192
channels=1
bus_width=8
address_mapping=rochrababgco
queue_structure=PER_BANK
refresh_policy=RANK_LEVEL_STAGGERED
row_buf_policy=OPEN_PAGE
cmd_queue_size=8
trans_queue_size=32

[power]
; IDD values from Micron DDR4-3200 8Gb x8 datasheet
VDD=1.2
IDD0=58
IDD3N=47
IDD4R=225
IDD4W=232
IDD5=280
IDD6=12

[other]
epoch_period=100000000
output_level=1
```

gem5

The gem5 simulator is a modular platform for computer-system architecture research, encompassing system-level architecture as well as processor microarchitecture. gem5 is a *community led* project with an open governance model.

+

About DRAMsim3

DRAMsim3 models the timing paramaters and memory controller behavior for several DRAM protocols such as DDR3, DDR4, LPDDR3, LPDDR4, GDDR5, GDDR6, HBM, HMC, STT-MRAM. It is implemented in C++ as an objected oriented model that includes a parameterized DRAM bank model, DRAM controllers, command queues and system-level interfaces to interact with a CPU simulator (GEM5, ZSim) or trace workloads. It is designed to be accurate, portable and parallel.

+

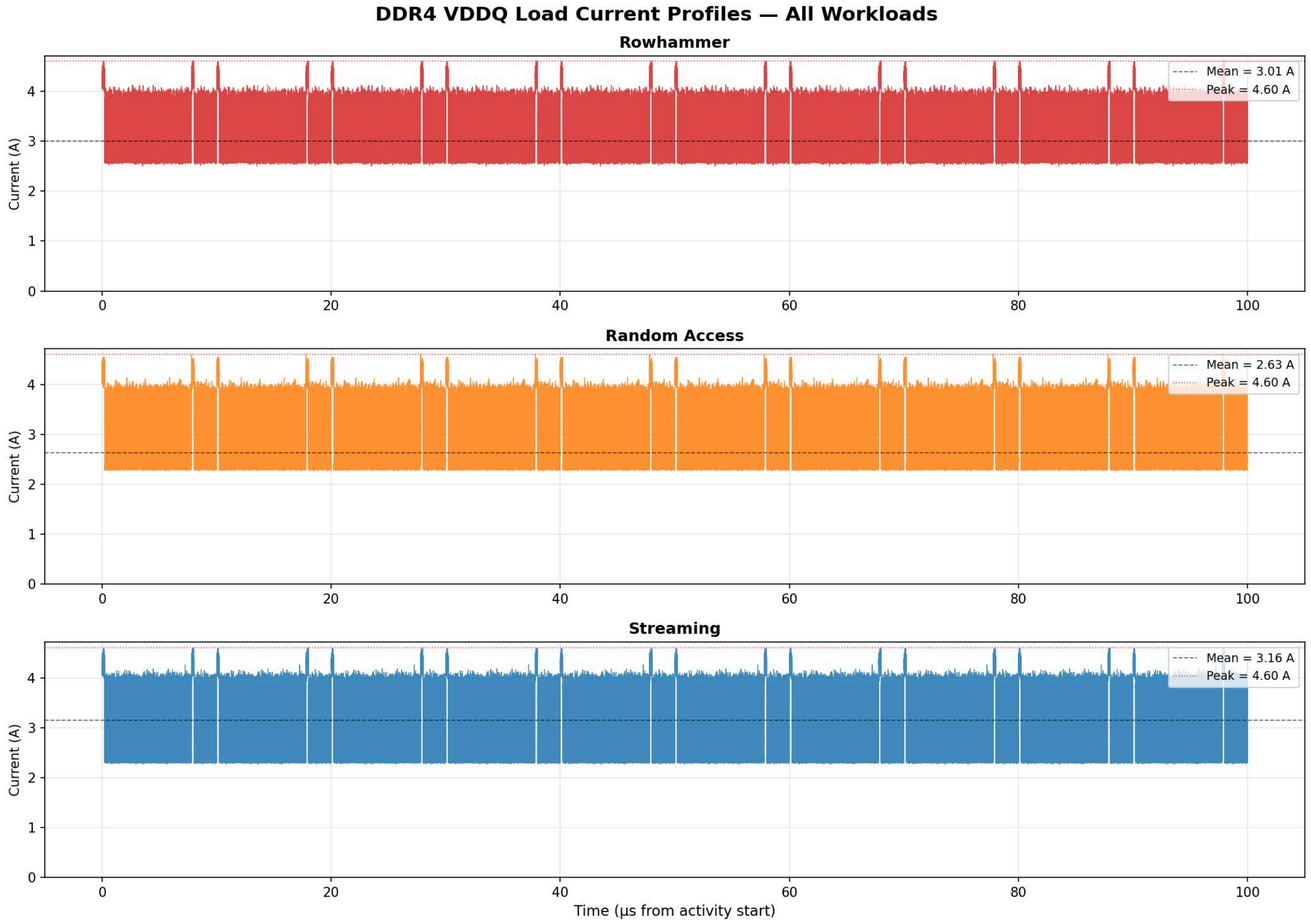
Code

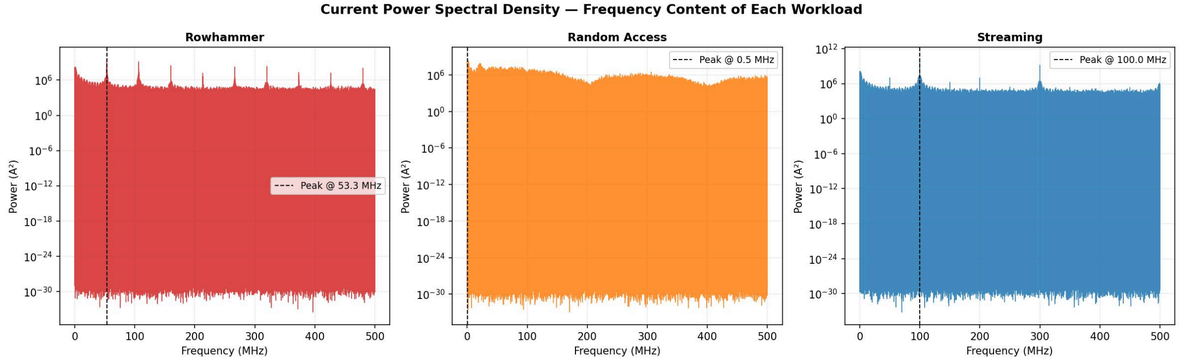
RAM Modelled: Micron MT40A1G8SA-062E

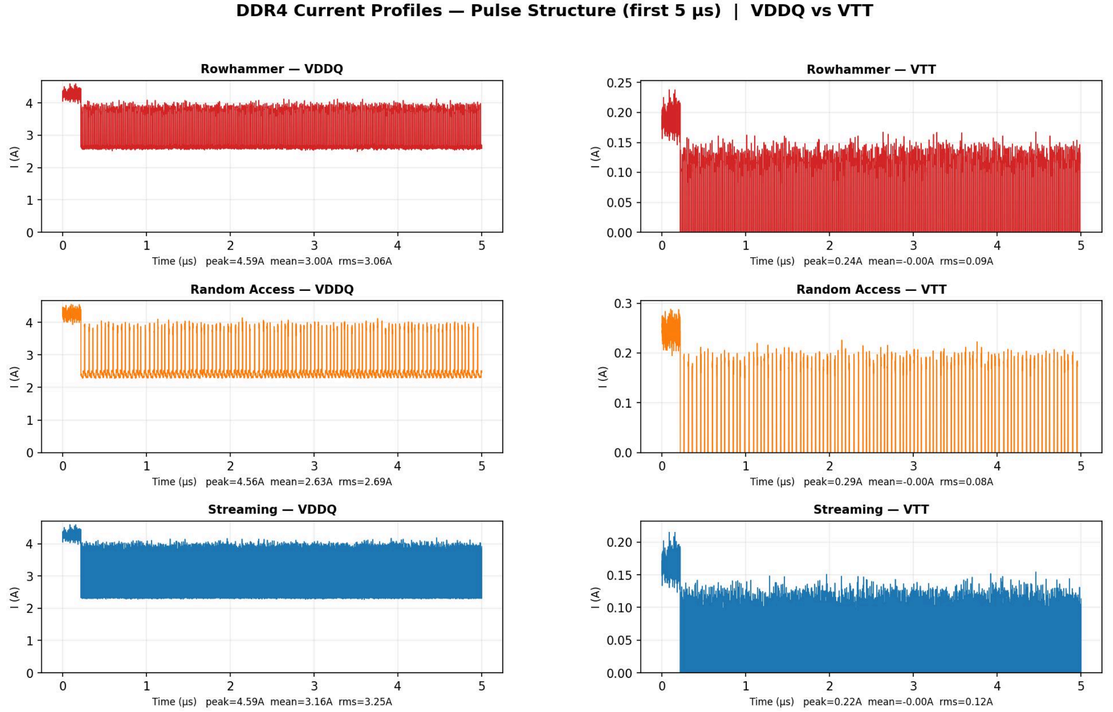
(Specs Found Online)

Current PWL Files

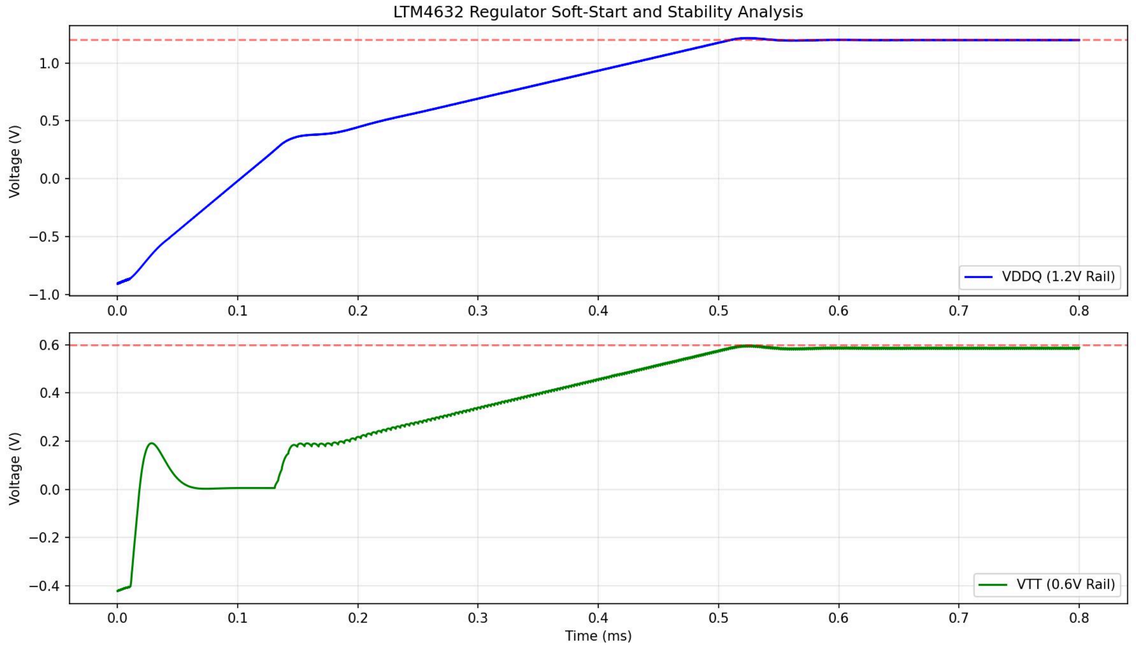
Results

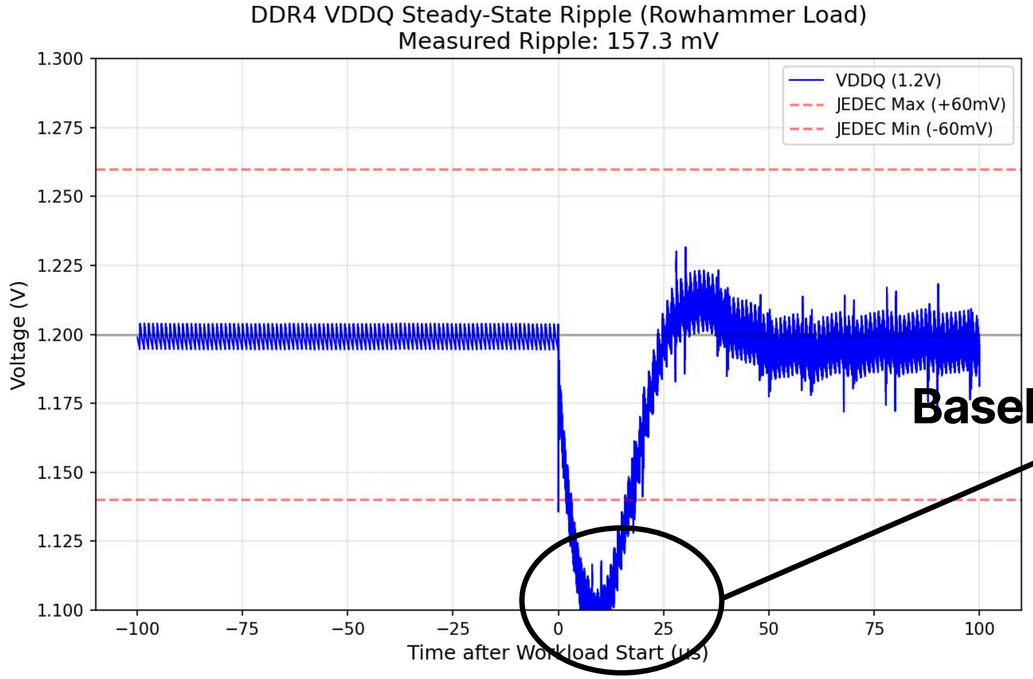

DDR4 VDDQ Load Current Profiles — All Workloads
Rowhammer
Mean = 3.01 A
Peak = 4.60 A
Random Access
Mean = 2.63 A
Peak = 4.60 A
Streaming
Mean = 3.16 A
Peak = 4.60 A
Current (A)
Time (µs from activity start)


Current Power Spectral Density — Frequency Content of Each Workload
Rowhammer
Peak @ 53.3 MHz
Random Access
Peak @ 0.5 MHz
Streaming
Peak @ 100.0 MHz
Power (A²)
Frequency (MHz)


DDR4 Current Profiles — Pulse Structure (first 5 µs) | VDDQ vs VTT
Rowhammer — VDDQ
Time (µs) peak=4.59A mean=3.00A rms=3.06A
Rowhammer — VTT
Time (µs) peak=0.24A mean=-0.00A rms=0.09A
Random Access — VDDQ
Time (µs) peak=4.56A mean=2.63A rms=2.69A
Random Access — VTT
Time (µs) peak=0.29A mean=-0.00A rms=0.08A
Streaming — VDDQ
Time (µs) peak=4.59A mean=3.16A rms=3.25A
Streaming — VTT
Time (µs) peak=0.22A mean=-0.00A rms=0.12A
I (A)

Results (Baseline)





Baseline Capacitor Network Fails

Results

Current PWL Files for Rowhammer Workload

+

Baseline Capacitor Network

+

cap_library.yaml

ddr4-pdn-optimizer > stage6_ga > cap_library.yaml

```yaml
capacitors:
  - part: "GRM155R60J106ME11"    # Murata 10uF 0402
    C_F:      10.0e-6
    ESR_ohm:  0.007
    ESL_H:    0.45e-9
    package:  "0402"

  - part: "GRM188R60J475ME19"    # Murata 4.7uF 0603
    C_F:      4.7e-6
    ESR_ohm:  0.012
    ESL_H:    0.87e-9
    package:  "0603"

  - part: "GRM033R60J104KE19"    # Murata 100nF 0201
    C_F:      100.0e-9
    ESR_ohm:  0.050
    ESL_H:    0.25e-9
    package:  "0201"

  - part: "GRM155R60J225ME47"    # Murata 2.2uF 0402
    C_F:      2.2e-6
    ESR_ohm:  0.010
    ESL_H:    0.45e-9
    package:  "0402"

  - part: "GRM155R60J105ME11"    # Murata 1uF 0402
```

+

Netlist

```
* DDR4 PDN Base Netlist for GA Optimization
V1 IN 0 12
R2 N001 0 60k
XU1 NC_01 VTT IN 0 MP_02 IN 0 NC_03 MP_04 VDDQ N004 VDDQ MP_05 MP_06 MP_07 MP_08 MP_09 VTTR
NC_10 MP_11 IN MP_12 MP_13 MP_14 MP_15 N002 N003 N001 LTM4632
C4 IN 0 4.7u x2 V=16 Irms=0 Rser=5m Lser=0
C3 IN 0 47u x2 V=25 Irms=0 Rser=5m Lser=0
C6 VDDQ N001 10p
C2 N003 0 .0001u
C9 VTTR 0 10n
Rload3 VTTR 0 75

* Bulk Capacitors (Fixed)
C7 VDDQ 0 100u V=25 Irms=0 Rser=5m Lser=0.87n
C8 VTT 0 100u V=25 Irms=0 Rser=5m Lser=0.87n

* Workload Current Sources
I1 VDDQ 0 PWL file="dummy.pwl"
I2 VTT 0 PWL file="dummy_vtt.pwl"

* AC Sources for Impedance Measurement (usually 0 in transient)
Iac_vddq VDDQ 0 AC 0
Iac_vtt VTT 0 AC 0

* Decoupling Network Placeholders
;DECOUPLING_NETWORK_VDDQ
;DECOUPLING_NETWORK_VTT

.options plotwinsize=0
.options numdgt=7
.options method=gear
.options cshunt=1e-15
.options reltol=0.005
.options trtol=7
.options abstol=1e-10
.tran .8m startup
.lib LTM4632.sub
.backanno
.end
```

→

Genetic Algorithm

Results

Fitness Function

- It heavily penalizes JEDEC voltage violations from the transient simulation, so illegal PDN designs are pushed out fast.
- It still rewards lower ripple even after a design passes, so the GA does not stop at “barely acceptable.”
- It penalizes impedance peaks above target in the AC sweep, which captures the frequency-domain PDN behavior that transient ripple alone would miss. This takes care of the unwanted anti-resonance
- It adds a soft BOM/area penalty so the GA does not just add more and more caps to win electrically.

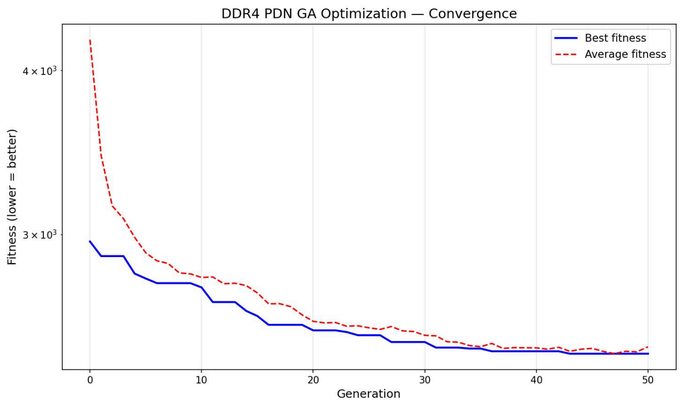
```
# Hard violation penalty (keeps GA away from illegal territory)
v_penalty = 0.0
if vddq_undershoot > VDDQ_TOL: v_penalty += (vddq_undershoot - VDDQ_TOL) * 8000
if vddq_overshoot  > VDDQ_TOL: v_penalty += (vddq_overshoot  - VDDQ_TOL) * 8000
if vtt_undershoot  > VTT_TOL:  v_penalty += (vtt_undershoot  - VTT_TOL)  * 8000
if vtt_overshoot   > VTT_TOL:  v_penalty += (vtt_overshoot   - VTT_TOL)  * 8000

# Continuous ripple quality — GA keeps improving even when already passing
# Scale: 1mV ripple reduction ≈ 1 unit improvement → meaningful signal
ripple_score = (vddq_ripple + vtt_ripple) * 1000
```

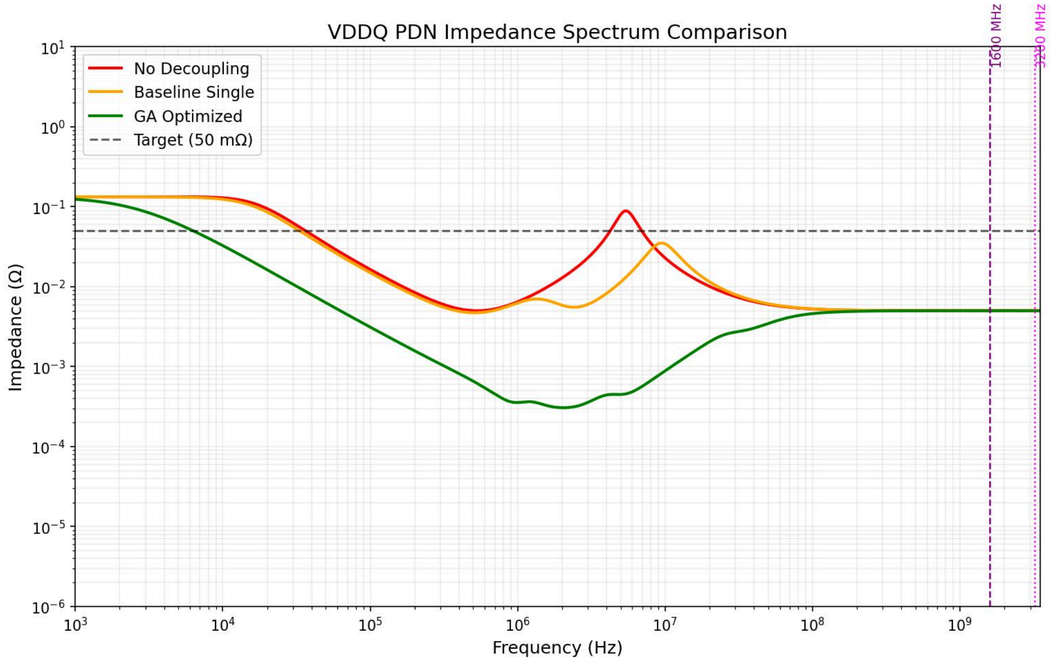
```
# --- 2. AC impedance (analytical — no LTSpice call) ---
res_ac     = _analytical_impedance(params)
mask       = res_ac['freq'] < FREQ_LIMIT
z_vddq_max = float(np.max(res_ac['z_vddq'][mask]))
z_vtt_max  = float(np.max(res_ac['z_vtt'][mask]))

# Raised to 3000× so impedance is taken seriously alongside voltage
ac_penalty = 0.0
if z_vddq_max > Z_TARGET_VDDQ: ac_penalty += (z_vddq_max - Z_TARGET_VDDQ) * 3000
if z_vtt_max  > Z_TARGET_VTT:  ac_penalty += (z_vtt_max  - Z_TARGET_VTT)  * 3000
```

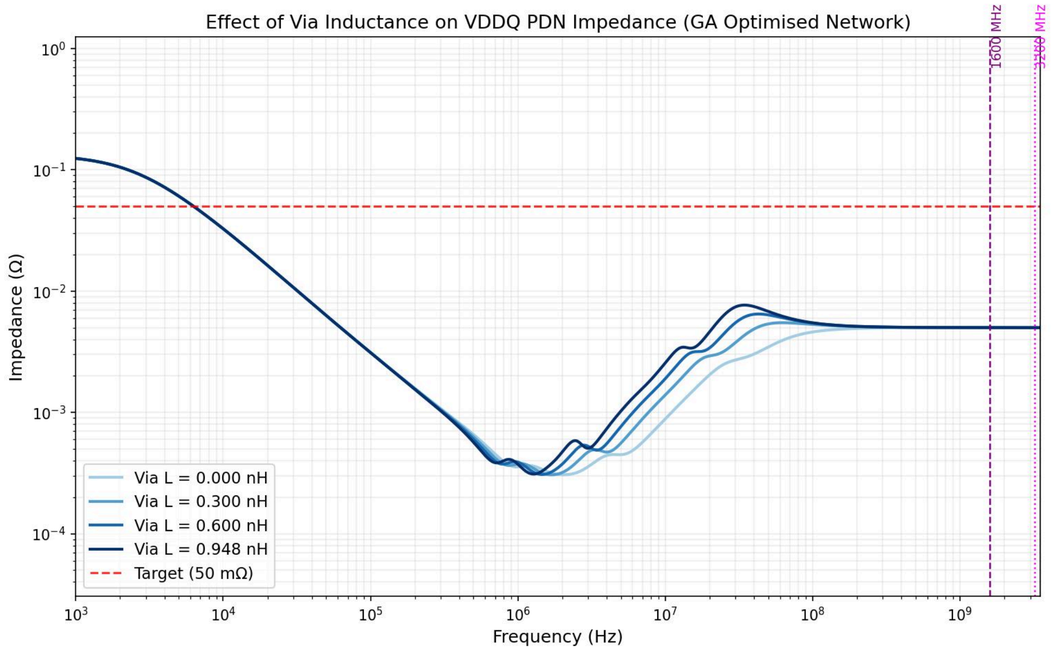
```
# --- 3. BOM / area penalty (relaxed ceiling, softer curve) ---
total_vddq = sum(params['vddq_network'])
total_vtt  = sum(params['vtt_network'])
area_penalty = 0.0
for count in (total_vddq, total_vtt):
    if count > MAX_CAPS_PER_RAIL:
        # Quadratic above ceiling — strongly discourages runaway growth
        area_penalty += (count - MAX_CAPS_PER_RAIL) ** 2 * 30
    elif count < MIN_CAPS_PER_RAIL:
        area_penalty += (MIN_CAPS_PER_RAIL - count) * 300

# Linear BOM cost: gentle pressure to prefer fewer caps when performance is equal
bom_penalty = (total_vddq + total_vtt) * 1.5
```

Results

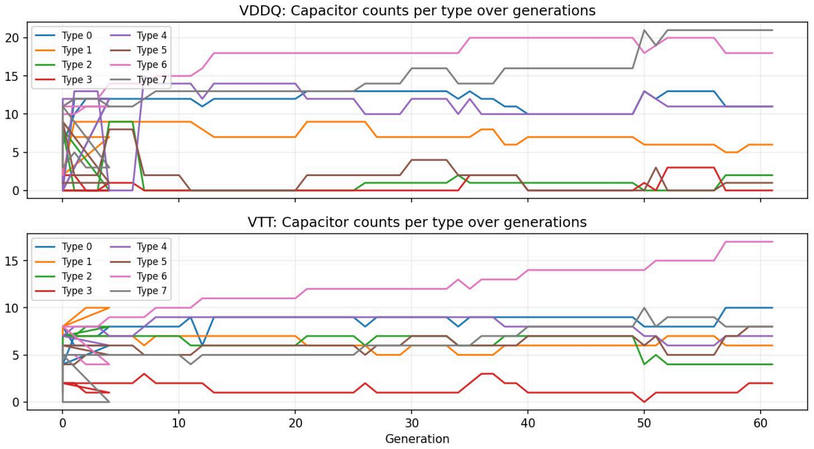
```
  Total wall time: 8.7 minutes

======================================================================
  OPTIMIZATION COMPLETE
======================================================================

=== Best Network Configuration ===
VDDQ Network:
  - 9x GRM155R60J106ME11 (10.00uF)
  - 8x GRM188R60J475ME19 (4.70uF)
  - 5x GRM033R60J104KE19 (0.10uF)
  - 10x GRM155R60J225ME47 (2.20uF)
  - 1x GRM033R60J474KE19 (0.47uF)
  - 7x GRM21BR60J226ME39 (22.00uF)
  - 11x GRM188R60J106ME47 (10.00uF)
VTT Network:
  - 5x GRM155R60J106ME11 (10.00uF)
  - 4x GRM188R60J475ME19 (4.70uF)
  - 6x GRM033R60J104KE19 (0.10uF)
  - 2x GRM155R60J225ME47 (2.20uF)
  - 3x GRM155R60J105ME11 (1.00uF)
  - 4x GRM033R60J474KE19 (0.47uF)
  - 5x GRM21BR60J226ME39 (22.00uF)
  - 6x GRM188R60J106ME47 (10.00uF)


Running Detailed Final Verification...

=== Performance Metrics (Detailed) ===
  VDDQ: Ripple 91.34mV  Z_peak 133.00mOhm  [PASS ✓]
  VTT: Ripple 54.21mV  Z_peak 133.00mOhm  [PASS ✓]
  BOM: 51 VDDQ + 35 VTT = 86 total caps
```

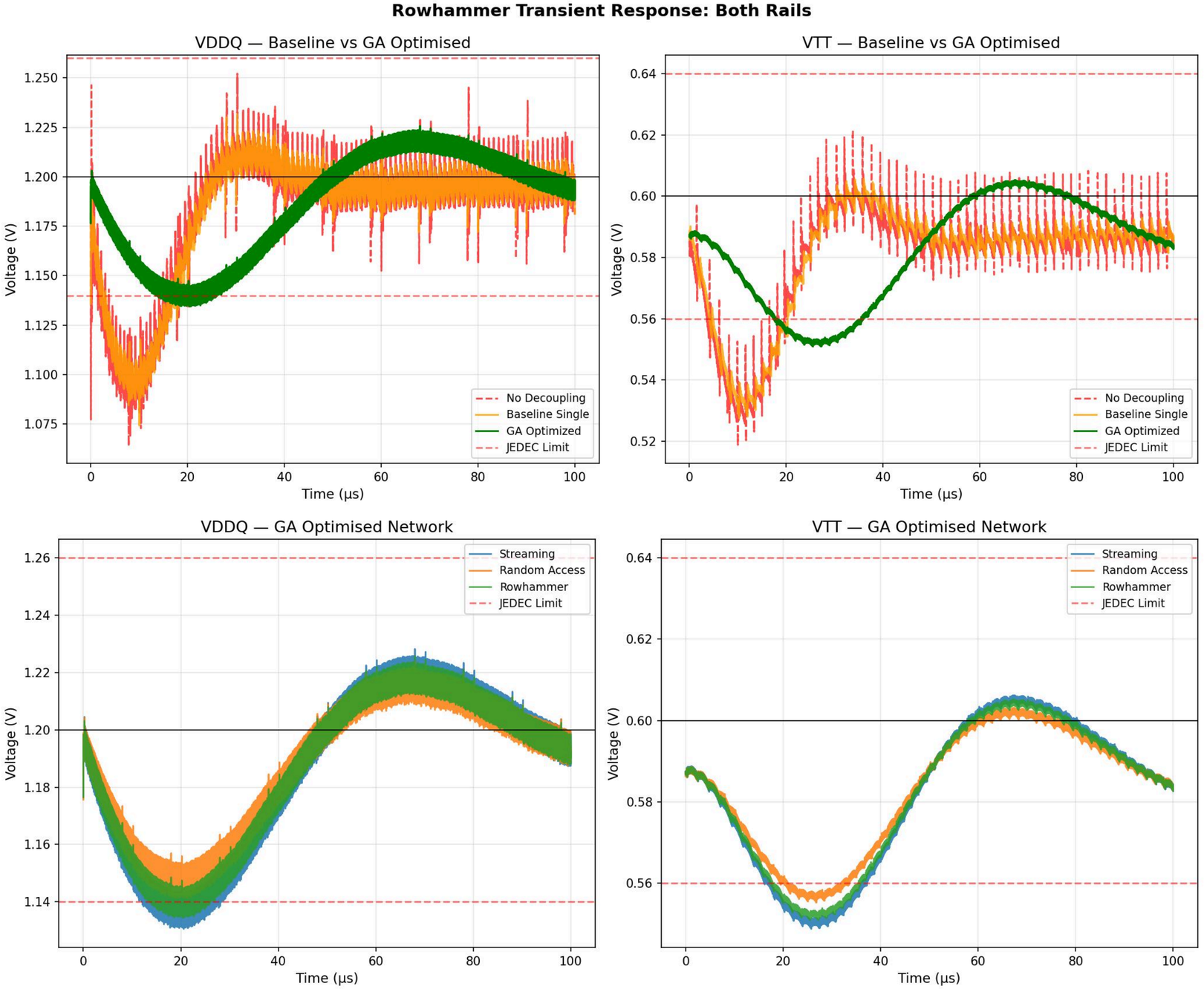






Results


VDDQ: Capacitor counts per type over generations
Type 0
Type 1
Type 2
Type 3
Type 4
Type 5
Type 6
Type 7
VTT: Capacitor counts per type over generations
Generation


Rowhammer Transient Response: Both Rails
VDDQ — Baseline vs GA Optimised
VTT — Baseline vs GA Optimised
VDDQ — GA Optimised Network
VTT — GA Optimised Network
Voltage (V)
Time (µs)
No Decoupling
Baseline Single
GA Optimized
JEDEC Limit
Streaming
Random Access
Rowhammer

References

Micron Technical Note TN-46-02
"Output Capacitor and Decoupling Design for DDR SDRAM."
Johnson, H. & Graham, M.
"High-Speed Digital Design: A Handbook of Black Magic." (via inductance calculation)
NXP/Freescale Application Note AN2582
"Hardware and Layout Design Considerations for DDR Memory Interfaces."
Rosen, Idan (Signal Integrity Journal)
"Contradicting the Common Belief: Decoupling Capacitors - Is More Always Better?" (Oct 2023).
Signal Integrity Journal
"On-Board Decoupling Capacitor Optimization."

Note: "The +-40mV limit for VTT is derived from the **JEDEC SSTL-2 specification**, which requires the termination rail to stay within 40mV of V_REF under most conditions, as cited in the Micron and NXP design guides."